

TSFS12 Hand-in 2: Planning for Vehicles with Differential Motion Constraints

MAXENCE MAURY

E-mail: maxma741@student.liu.se

LiU-ID: maxma741

JESSY SANFILIPPO

E-mail: jessa070@student.liu.se

LiU-ID: jessa070

Contents

1	STATE LATTICE PLANNING	1
1.1	Exercise 4.1: Motion Primitives	1
1.2	Exercise 4.2: Lattice Planner Implementation	2
1.3	Exercise 4.3: Experiments and Heuristics	3
2	RRT WITH PARTICLE MODEL	6
2.1	Exercise 4.4: Implementation of RRT	6
2.2	Exercise 4.5: Path Planning Results	7
2.3	Exercise 4.6: Parameter Analysis	9
3	RRT WITH KINEMATIC CAR MODEL	10
3.1	Exercise 4.7: Implementation of RRT with differential constraints	10
3.2	Exercise 4.8: Path Planning Results	10
3.3	Exercise 4.9: Experiments with Parameters	11
3.4	Exercise 4.10: Distance Function Variants	11
3.5	Exercise 4.11: Control Set Discretization	12
4	CONCLUSION	13

1 STATE LATTICE PLANNING

1.1 Exercise 4.1: Motion Primitives

- We use the kinematic single-track model:

$$\dot{x} = v \cos(\theta), \quad (1)$$

$$\dot{y} = v \sin(\theta), \quad (2)$$

$$\dot{\theta} = \frac{v}{L} \tan(\delta), \quad (3)$$

where (x, y) is the position of the rear axle, θ the heading, L the wheelbase, v the constant forward velocity, and δ the steering angle.

- The function `compute_mprims` generates motion primitives by solving optimal control problems for transitions between discrete states. It takes as arguments:

```
compute_mprims(theta_init, lattice, L, v, u_max)
```

and returns optimized trajectories saved in `mprims.pickle`. The function exploits symmetry to reduce the number of required optimizations.

- Generated motion primitives cover forward driving for a discrete set of orientations

$$\theta \in \left\{ -\frac{3\pi}{4}, -\frac{\pi}{2}, -\frac{\pi}{4}, 0, \frac{\pi}{4}, \frac{\pi}{2}, \frac{3\pi}{4}, \pi \right\},$$

with reverse primitives obtained automatically by symmetry. Figure 1 illustrates the resulting set.

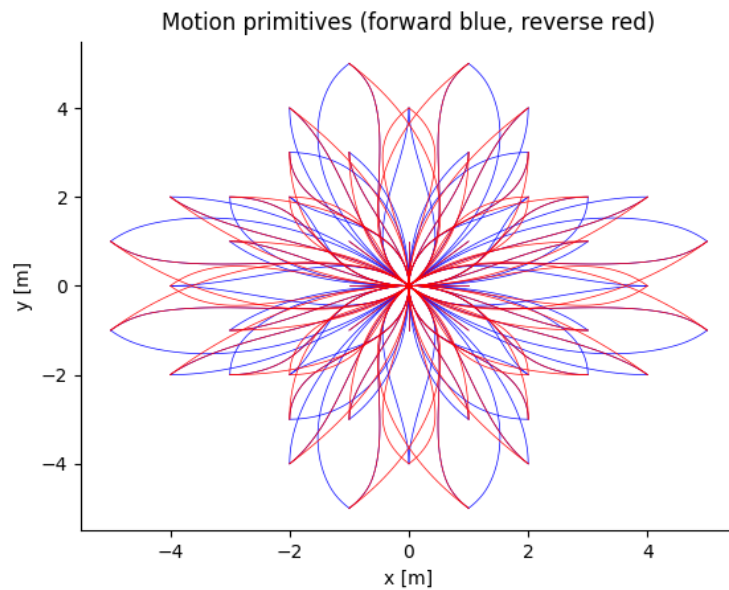


Figure 1: Computed motion primitives for the different orientations. Blue: forward, red: reverse (obtained by symmetry).

1.2 Exercise 4.2: Lattice Planner Implementation

- We implemented the planners `depth_first`, `breadth_first`, `dijkstra`, `best_first`, and `astar`, reusing our code from Hand-in 1. The `next_state` function provides valid successors by checking collisions and lattice constraints.
- Each planner was run on Mission 5. The resulting paths are shown in Figure 2.

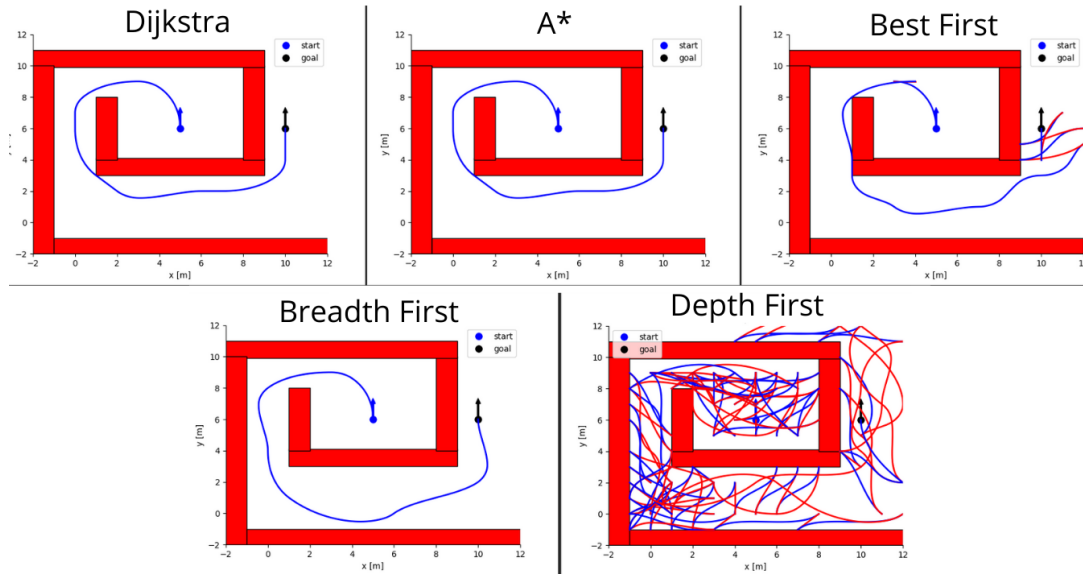


Figure 2: Planned trajectories for Mission 5 with different planners. Blue: forward motion, red: reverse motion.

- Comparison of results:

- **Depth-First Search (DFS):** Explores deeply without regard for cost. Produces long and suboptimal paths, often with excessive backtracking.
- **Breadth-First Search (BFS):** Guarantees shortest path in number of steps (discrete transitions), but not in continuous path length. Still, it produces a relatively efficient solution.
- **Dijkstra:** Expands nodes by accumulated cost. Finds the globally optimal path with respect to path length. Computation time is higher than BFS but ensures optimality.
- **Best-First Search:** Expands nodes closest to the goal (heuristic only). It is fast but often produces suboptimal paths, as seen by the detours.
- **A*:** Balances accumulated cost with heuristic. Produces nearly optimal results but expands fewer nodes than Dijkstra, making it more efficient.

Overall, Dijkstra guarantees the shortest path, A* provides a near-optimal solution faster, while DFS and Best-First can be very inefficient in complex environments.

1.3 Exercise 4.3: Experiments and Heuristics

- We tested multiple missions by varying obstacles and start/goal states. Results showed that plan length increases significantly with obstacle density, and planners differ in the number of expanded nodes.
- Plots of plan length vs. computation time (see Figure 3) and planning time vs. expanded nodes (see Figure 4) confirmed that A* provides the best compromise: it consistently requires fewer expansions than Dijkstra while yielding nearly identical path length.

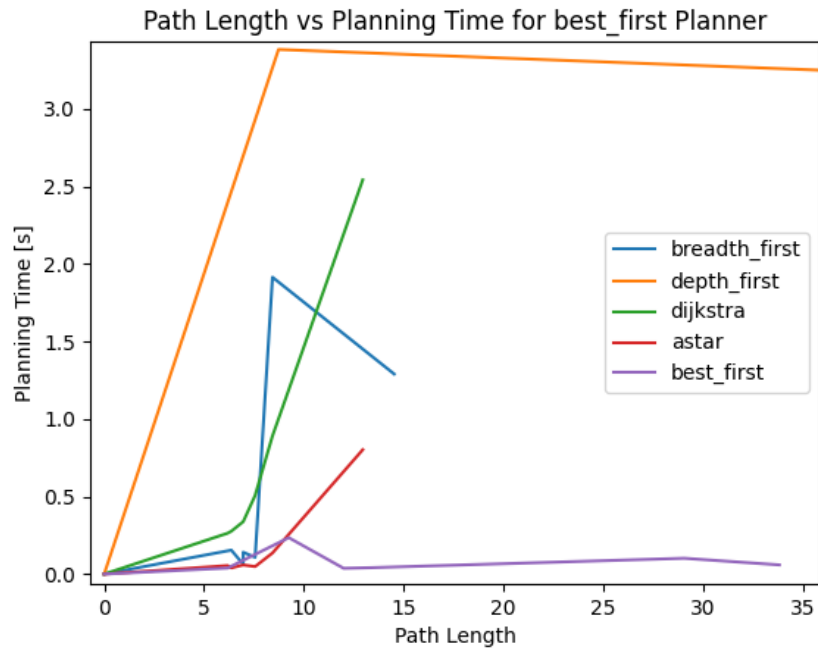


Figure 3: Plan length vs. computation time

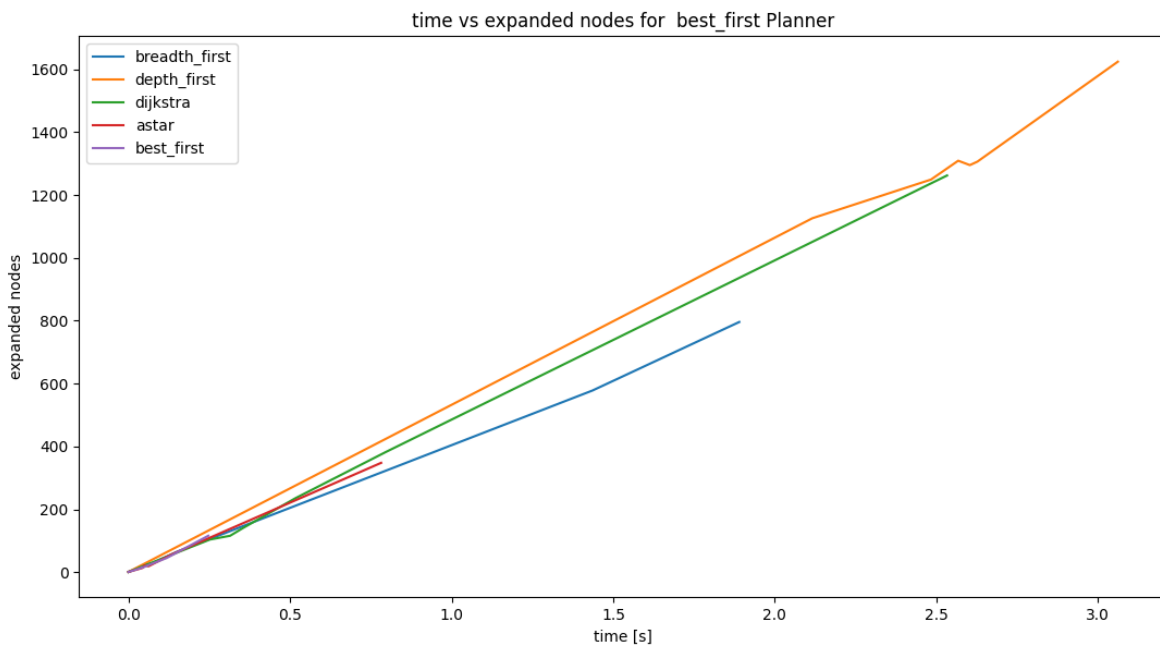


Figure 4: Planning time vs. expanded nodes

- Different heuristics for `cost_to_go` were compared:
 - Euclidean distance: simple and effective.
 - Manhattan distance: less effective due to angular constraints.
 - Weighted heuristics penalizing orientation mismatch: reduced expansions in some missions.
 - Zero Heuristic.

Figure 5 shows the different computation times for Dijkstra, A* and best first planners for different heuristic implementations. We can see that the best heuristic is weighted orientation mismatch and the worst is zero heuristic with 35 seconds for best first. Euclidean and Manhattan performed similarly with a slightly better time for Manhattan heuristic.

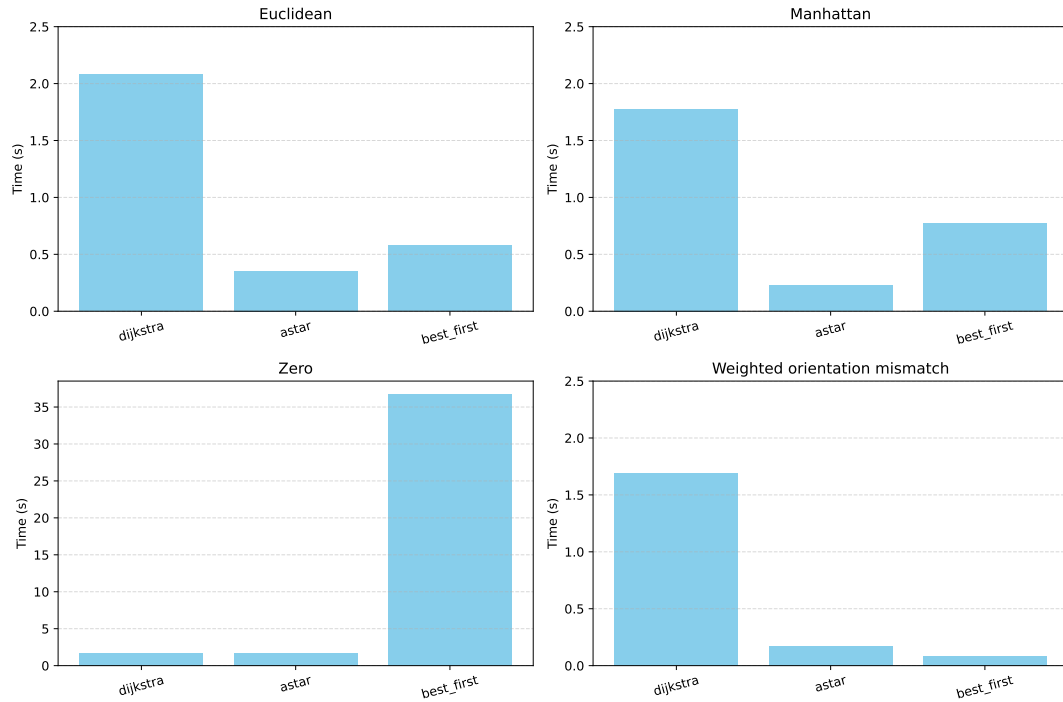


Figure 5: Time comparison for different heuristics

- **Reflections:**

- Dijkstra's result is optimal but expensive.
- A* offers a good trade-off: better performances and same optimality.
- BFS can be useful in small worlds, but scales poorly.
- Heuristic design has a strong impact on efficiency.

2 RRT WITH PARTICLE MODEL

2.1 Exercise 4.4: Implementation of RRT

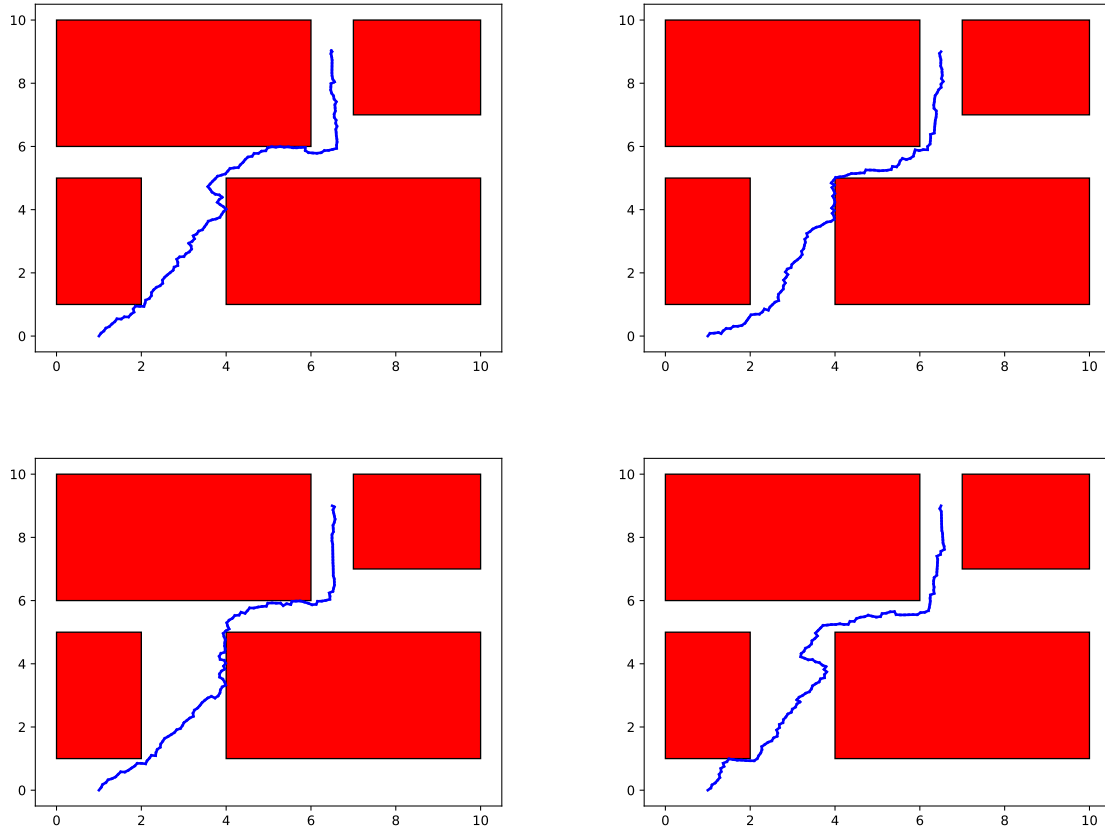


Figure 6: examples of RRT plannings

- We completed the function `rrt_particle(start, goal, world, opts)`. The algorithm builds a tree incrementally:
 1. Sample a random state x_{rand} from the free space, with probability β replace it by the goal (goal bias).
 2. Find the nearest existing node x_{near} .
 3. Steer from x_{near} towards x_{rand} by step size λ , obtaining x_{new} .
 4. If the line segment $(x_{\text{near}}, x_{\text{new}})$ is collision-free, add x_{new} to the tree.
 5. Stop if $\|x_{\text{new}} - x_{\text{goal}}\| < \epsilon$ or after K iterations.
- The implementation supports the following parameters:

β = goal bias, λ = step size, ϵ = goal threshold, K = maximum iterations.
- The tree is stored as a list of nodes and a parent array for lightweight backtracking of paths.

2.2 Exercise 4.5: Path Planning Results

- Using the completed `rrt_particle` planner, we obtained feasible paths in the provided world. An example is shown in Figure 7.

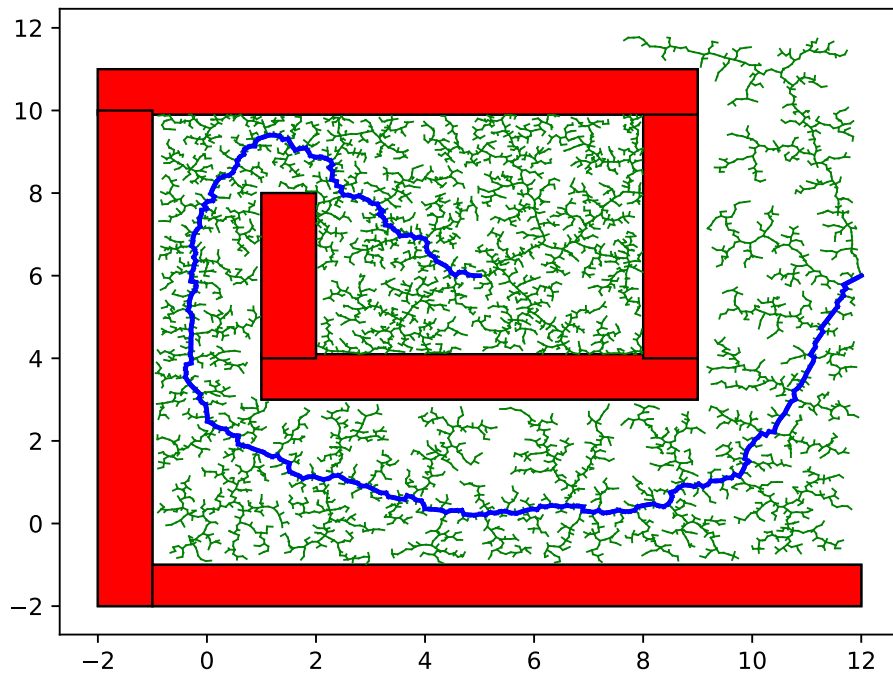


Figure 7: RRT tree (gray) and final solution path (blue) for the particle model. Obstacles are shown in red, start (green) and goal (black).

- Observations:
 - The RRT tree tends to rapidly explore the space, covering free regions uniformly.
 - The final solution depends on randomness and may differ in each run.
 - In cluttered environments, many branches terminate in collisions before reaching the goal.
- With different obstacle configurations:
 - Open environments lead to very straight paths.
 - Narrow passages require more nodes and iterations to succeed, highlighting the probabilistic completeness of RRT.

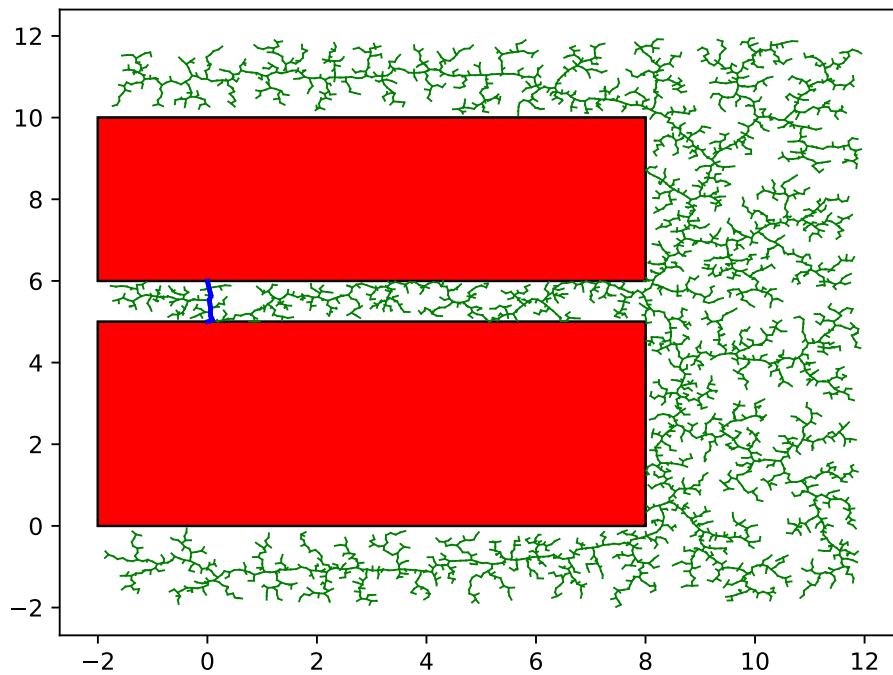


Figure 8: RRT tree (green) and final solution path (blue) for the particle model. Obstacles are shown in red.

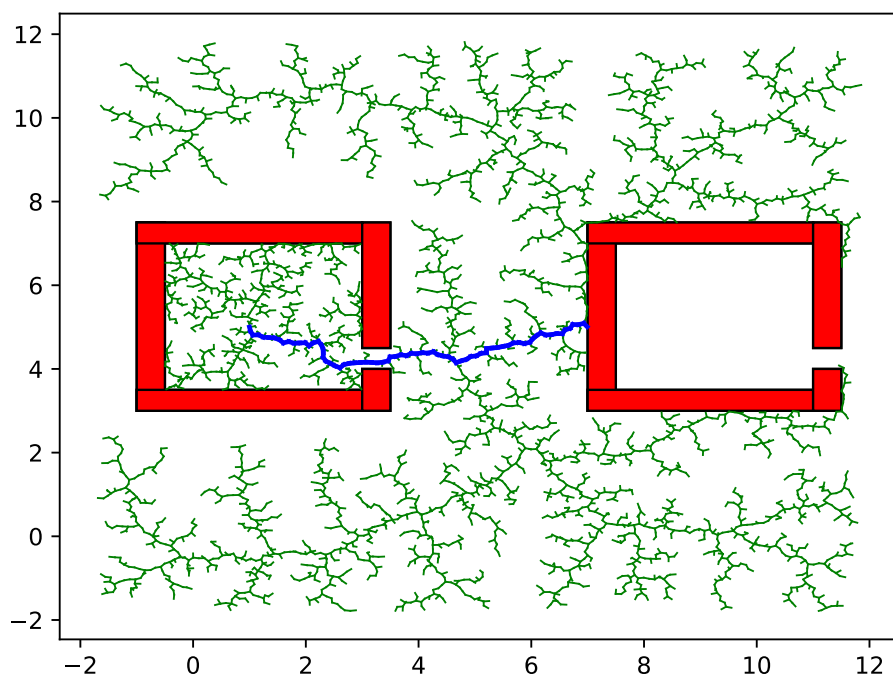


Figure 9: RRT on a mission with narrow passages

2.3 Exercise 4.6: Parameter Analysis

- **Step size λ :** Small λ produces dense trees with smooth paths but requires more iterations. That's why on small labmda simulations, there are way more nodes explored, but a smaller path length. Large λ accelerates exploration but may miss narrow passages. It may lead to less nodes explored, but greater path length.
- **Goal bias β :** Increasing β improves convergence speed when the goal is in an open area because RRT explores less and goes in the right direction. Too high β can reduce exploration diversity.
- **Threshold ϵ :** Larger ϵ allows earlier termination, but paths may end further from the goal. Small ϵ improves accuracy but increases iterations.
- Figure 10 summarizes the influence of parameters.

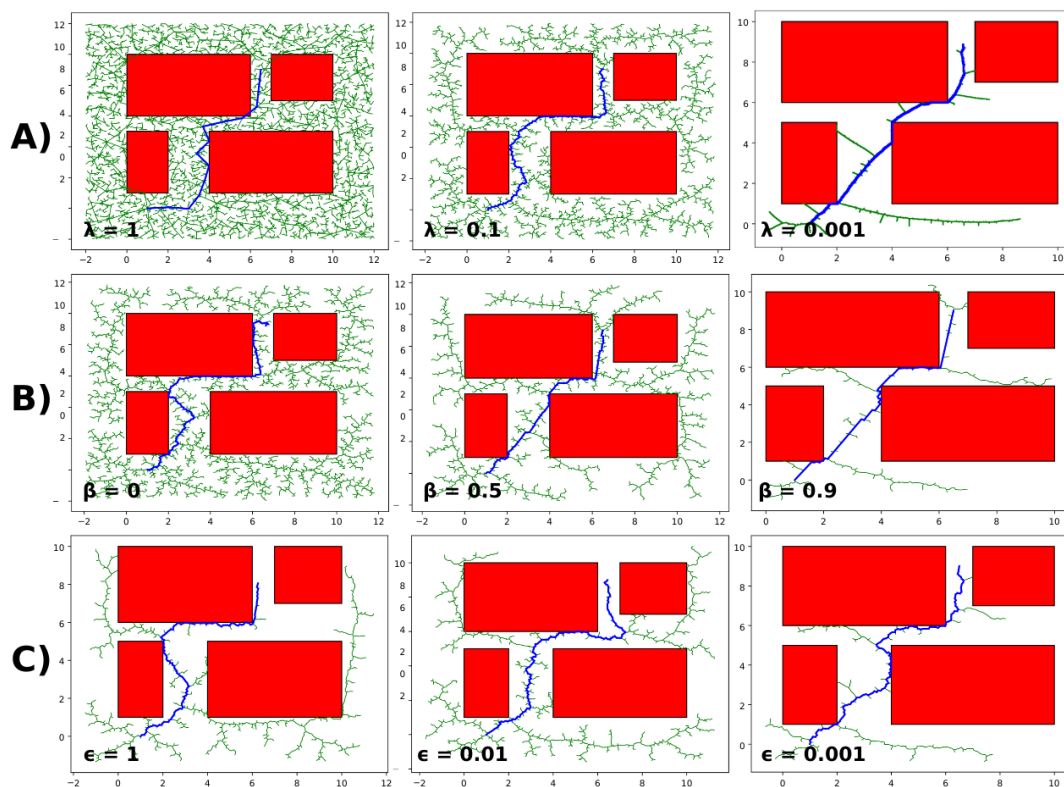


Figure 10: Effect of RRT parameters: (a) varying step size λ , (b) varying goal bias β , (c) varying threshold ϵ .

- **Reflections:**
 - There is a trade-off between exploration (large λ , low β) and exploitation (small λ , high β).
 - For practical planning, moderate values (e.g. $\lambda = 0.1$, $\beta = 0.5$, $\epsilon = 0.01$) provide robust performance.
 - The number of nodes in the final path grows with decreasing λ and ϵ , but so does computation time.

3 RRT WITH KINEMATIC CAR MODEL

3.1 Exercise 4.7: Implementation of RRT with differential constraints

- We adapted the particle RRT to handle the kinematic car model:

$$\dot{x} = v \cos(\theta), \quad (4)$$

$$\dot{y} = v \sin(\theta), \quad (5)$$

$$\dot{\theta} = \frac{v}{L} \tan(\delta), \quad (6)$$

where the control input is $\delta \in U$, a discretized set of steering angles.

- The function `rrt_diff(start, goal, u_c, sim, world, opts)` was implemented. The `sim_car` function simulates trajectories with step size λ using Euler integration.
- Each edge in the tree is associated with:
 - a control input $\delta \in U$,
 - a trajectory $(x(t), y(t), \theta(t))$ stored in `state_trajectories`.

3.2 Exercise 4.8: Path Planning Results

- The planner was run in the provided example world. Figure 11 shows the resulting tree and solution path.

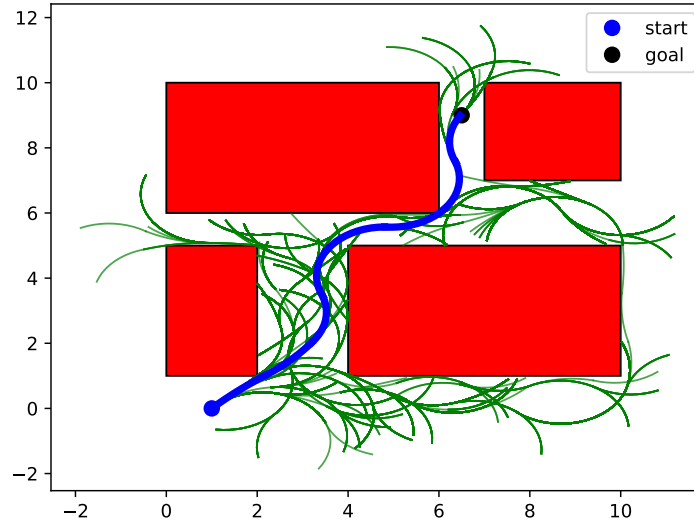


Figure 11: RRT with kinematic car model. Green: tree, blue: solution path, red: obstacles.

- The solution is obtained by backtracking through the parent array, retrieving both states and control inputs. Unlike the particle model, edges are curved, reflecting the nonholonomic constraints.

3.3 Exercise 4.9: Experiments with Parameters

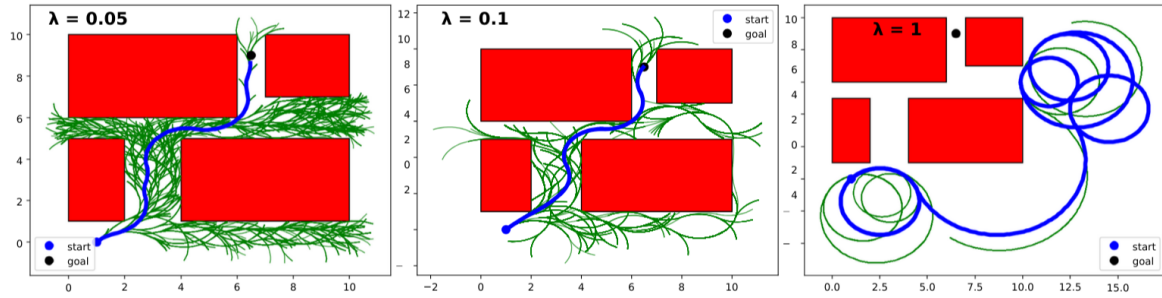


Figure 12: RRT with kinematic car model. Green: tree, blue: solution path, red: obstacles.

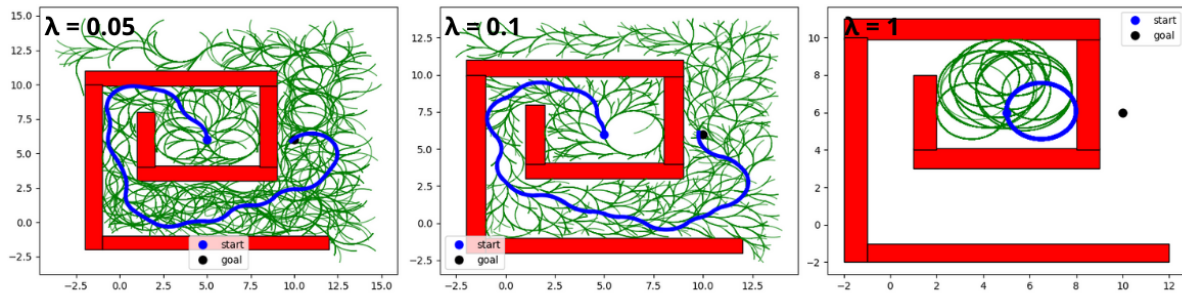


Figure 13: RRT with kinematic car model. Green: tree, blue: solution path, red: obstacles.

- Varying the time step λ :
 - Small λ : denser tree, finer control, longer computation.
 - Large λ : faster exploration, risk of collisions and coarse trajectories.
- Experiments in different environments confirmed that smaller λ improves feasibility in narrow passages, but increases path length.
- The solution path length and number of nodes both depend strongly on λ .

3.4 Exercise 4.10: Distance Function Variants

- We tested different distance measures in `distance_fcn`:
 1. Euclidean distance in (x, y, θ) directly.
 2. Weighted distance:

$$d((x, y, \theta), (x', y', \theta')) = \|(x, y) - (x', y')\| + w \cdot |\theta - \theta'|,$$

with $w > 0$ to penalize orientation mismatch.

3. Angular wrap-around to avoid discontinuities near $\pm\pi$.

- Results:
 - Pure Euclidean distance caused poor goal convergence when orientation mattered.
 - Weighted distance improved efficiency, reducing unnecessary expansions.
 - Proper angle wrapping was essential for stable performance.

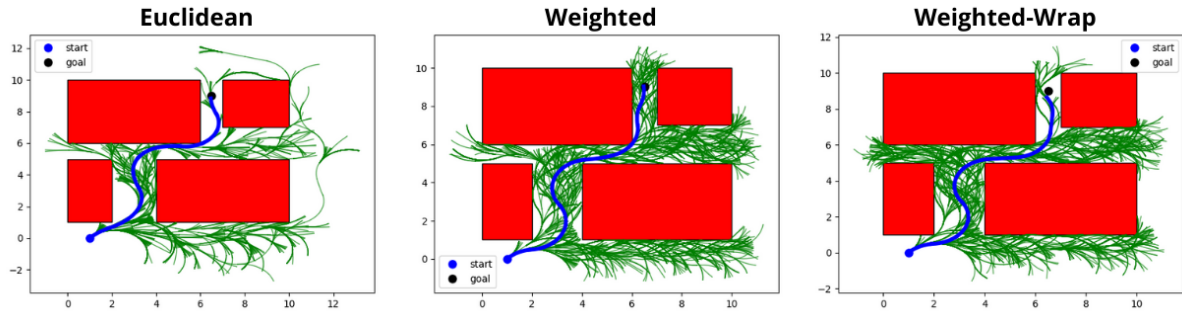


Figure 14: Different `distance_fcn` implementations

3.5 Exercise 4.11: Control Set Discretization

Since the beginning we used $|U| = 11$.

- The control set U was varied from coarse (3 angles) to fine (21 angles).
- Results:
 - Small $|U|$: tree is sparse, some goals unreachable.
 - Large $|U|$: better coverage, more diverse paths, but higher computation time.
- Figure 15 illustrates the effect.

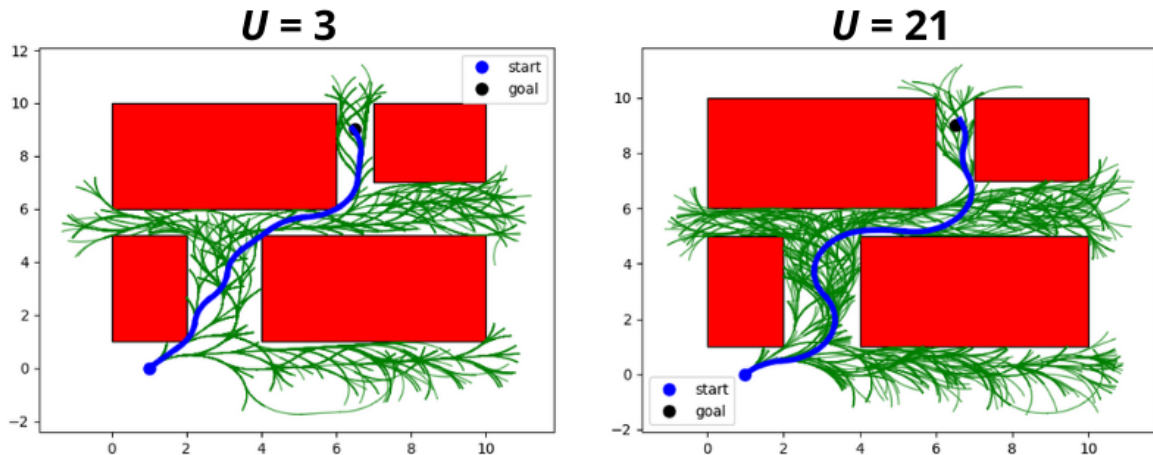


Figure 15: Effect of control discretization. Left: $|U| = 3$, Right: $|U| = 21$. Finer discretization improves reachability but increases complexity.

- **Reflections:** A moderate discretization (e.g. $|U| = 11$) provides a good balance between computational cost and path feasibility.

4 CONCLUSION

- **State Lattice Planning:** Guaranteed feasibility with motion primitives that respect car kinematics. Dijkstra ensures optimality but is computationally heavy, while A* balances speed and path quality. Heuristics strongly influence performance.
- **RRT (Particle and Car Models):** Flexible and probabilistically complete. RRT for particles quickly explores the space but yields suboptimal paths. With car constraints, the tree captures feasible motion but requires careful tuning of time step and distance metrics.
- **RRT*:** Extends RRT by including cost-to-come and rewiring. Provides asymptotic optimality, producing paths that improve with iterations. However, it increases computational demands.
- **Trade-offs:**
 - **Feasibility vs. Optimality:** RRT focuses on feasibility; lattice planning and RRT* target optimality.
 - **Computation vs. Accuracy:** Faster planners (DFS, RRT) yield quick but suboptimal solutions; optimal planners (Dijkstra, RRT*) require more resources.
- **Which planner to use?**
 - For environments with strong kinematic constraints and when path quality is crucial: **Lattice planning with A* or Dijkstra.**
 - For high-dimensional or unstructured environments: **RRT.**
 - When optimality is important and time allows: **RRT*.**